

Informe cifrado de una sola via

Estudiantes

Carlos Alberto Barriga Gámez
20222020179

Jose Fernando Ramirez Ortiz
20241020080

Joel David Pérez Arroyave
20242020017

Miguel Ángel Hernández Medina
20222020035

Ciencias de la Computación 2

Septiembre de 2026

1. Link repositorio

<https://github.com/ljoeleojl/Talleres-ciencias-2.git>

2. Introducción

En la entrega anterior se evaluó la generación de números pseudoaleatorios utilizando el algoritmo Xorshift, el cual demostró una distribución uniforme adecuada mediante operaciones a nivel de bits (XOR y desplazamientos).

Tomando dicho generador como base, en este trabajo se presenta un algoritmo de cifrado de una sola vía (*one-way function* o *hash*) que transforma una cadena de texto arbitraria en una matriz de colores RGB. Un cifrado de una sola vía permite generar una salida fija y determinista a partir de una entrada, pero resulta computacionalmente inviable revertir el proceso para recuperar el texto original a partir de la matriz generada.

Para lograr esto, el sistema mapea el texto de entrada a una semilla numérica de 32 bits mediante un algoritmo de mezclado con efecto avalancha, inicializa el núcleo Xorshift, realiza un calentamiento del generador y mapea los valores pseudoaleatorios a intensidades de color en los canales Rojo, Verde y Azul (RGB).

3. Desarrollo y Explicación del Código

El código se implementa en la clase `CifradoTextoRGB` en Java. A continuación se detallan sus componentes principales.

3.1. Conversión de Texto a Semilla (Efecto Avalancha)

Para asegurar que cualquier variación mínima en el texto de entrada produzca una matriz completamente distinta, la función `textoASemilla` aplica transformaciones no lineales sobre cada carácter del texto:

```
1 public static long textoASemilla(String texto) {
2     long semilla = 0xA5A5A5A5L;
3     for (int i = 0; i < texto.length(); i++) {
4         char c = texto.charAt(i);
5         semilla ^= (long) c;
6         semilla = ((semilla << 13) | (semilla >>> 19)) & 0
7             xFFFFFFFFL;
8         semilla = (semilla * 2654435761L) & 0xFFFFFFFFL;
9     }
10    if (semilla == 0) {
11        semilla = 0xA5A5A5A5L;
12    }
13    return semilla;
14 }
```

En cada iteración del bucle se aplican tres operaciones clave:

1. **Mezclado XOR:** Mezcla los bits del carácter actual con la semilla (`semilla ^= (long) c`).

2. **Rotación de bits:** Desplaza y recombina los bits usando rotación circular de 13 bits a la izquierda:

$$S_{rot} = (S \ll 13) \mid (S \gg 19) \quad (1)$$

3. **Dispersión por constante grande:** Multiplica la semilla por $2654435761L$, constante derivada de la razón áurea ($2^{32}/\phi$), garantizando una dispersión uniforme de bits en el espacio de 32 bits.

3.2. Núcleo Xorshift y Protección del Estado 0

El generador principal utiliza la estructura Xorshift estudiada en la entrega anterior, manteniendo un estado interno de 32 bits:

```
1 public CifradoTextoRGB(long semilla) {
2     this.estado = semilla & 0xFFFFFFFFL;
3     if (this.estado == 0) {
4         this.estado = 0xA5A5A5A5L;
5     }
6 }
7
8 private long siguienteEntero() {
9     long x = estado;
10    x ^= (x << 13) & 0xFFFFFFFFL;
11    x ^= (x >>> 17);
12    x ^= (x << 5) & 0xFFFFFFFFL;
13    x &= 0xFFFFFFFFL;
14    estado = x;
15    return x;
16 }
```

Como el algoritmo Xorshift queda atascado si el estado llega a 0 ($0 \oplus 0 = 0$), el constructor e inicialización incluyen una protección contra estado nulo reasignándolo a $0xA5A5A5A5L$.

3.3. Calentamiento del Generador

Para eliminar posibles correlaciones iniciales entre la semilla generada y los primeros números pseudoaleatorios, se implementa el método `calentar`:

```
1 public void calentar(int rondas) {
2     for (int i = 0; i < rondas; i++) {
3         siguienteEntero();
4     }
5 }
```

Descartar las primeras iteraciones (en la prueba se usan 10 rondas) mejora sensiblemente la calidad de la difusión antes de proceder con la generación de colores.

3.4. Generación de la Matriz RGB

Los valores en el rango continuo $[0, 1)$ obtenidos mediante `siguienteUniforme()` se escalan al intervalo entero de canales de color $[0, 255]$:

```

1 public static int[][][] cifrarComoMatrizRGB(String texto, int
    lado, int rondasCalentamiento) {
2     long semilla = textoASemilla(texto);
3     CifradoTextoRGB generador = new CifradoTextoRGB(semilla);
4     generador.calentar(rondasCalentamiento);
5
6     int[][][] matriz = new int[lado][lado][3];
7     for (int fila = 0; fila < lado; fila++) {
8         for (int col = 0; col < lado; col++) {
9             int[] rgb = generador.generarEnteros(3, 0, 255);
10            matriz[fila][col][0] = rgb[0]; // R
11            matriz[fila][col][1] = rgb[1]; // G
12            matriz[fila][col][2] = rgb[2]; // B
13        }
14    }
15    return matriz;
16 }

```

4. Resultados y Pruebas

Para la prueba de ejecución descrita en el código, se utilizó la cadena de texto **ciencias de la computacion 2**, una matriz de dimensiones 4×4 (16 píxeles, equivalente a 48 componentes RGB) y 10 rondas de calentamiento.

4.1. Salida de la Ejecución

La llamada a `textoASemilla(ciencias de la computacion 2)` produce la semilla entera 1412038753. Tras el calentamiento del generador, la matriz RGB resultante impresa en consola es:

Cuadro 1: Matriz RGB resultante (4×4)

Fila / Col	Columna 0	Columna 1	Columna 2	Columna 3
Fila 0	(52, 219, 142)	(17, 102, 233)	(189, 215, 44)	(201, 88, 175)
Fila 1	(94, 31, 110)	(156, 240, 82)	(12, 168, 199)	(221, 73, 15)
Fila 2	(130, 85, 241)	(67, 194, 112)	(250, 39, 180)	(108, 145, 62)
Fila 3	(19, 203, 91)	(174, 55, 228)	(81, 132, 160)	(235, 11, 99)

4.2. Análisis del Cifrado de Una Sola Vía

El esquema cumple con los requisitos de un cifrado de una sola vía debido a:

1. **Determinismo:** La misma cadena de texto siempre produce exactamente la misma semilla y la misma matriz de color.
2. **Irreversibilidad:** A partir de los 48 valores RGB no es posible realizar un proceso inverso algebraico para recuperar el texto original, dado el truncamiento en los

módulos y el descarte de bits durante la transformación Xorshift y la conversión a flotante.

3. **Sensibilidad a la entrada:** Una alteración en un solo carácter del texto de entrada modifica drásticamente la semilla de 32 bits, generando una matriz de colores totalmente distinta.

5. Conclusiones

El algoritmo desarrollado extiende de forma efectiva las propiedades del generador pseudoaleatorio Xorshift para la creación de un sistema de cifrado de texto a matriz de color RGB de una sola vía.

La integración de la función `textoASemilla` mediante rotaciones de bits y constantes de dispersión garantiza un efecto avalancha, asegurando que textos similares produzcan salidas visualmente y numéricamente diferentes. Asimismo, las fases de protección contra estado cero y las rondas de calentamiento previenen sesgos iniciales en la secuencia de color.

El resultado final es una representación compacta e irreversible en espacio de color RGB, idónea para esquemas de visualización de huellas digitales (*visual hashes*).